

HW3_Report



Author: 111511198 鄭恆安

Concept and Code Details of the Decision Tree

Model Overview

The `DecisionTreeClassifier` constructs a binary decision tree to classify data using a chosen impurity measure (`entropy` or `gini`). It recursively splits data to maximize information gain or gain ratio, depending on the `method` parameter, until it reaches either a maximum depth or a minimum sample requirement per leaf node.

TreeNode Class

- Represents each node in the tree, which can be either a "leaf" or "node"
 - if it is "node", it will contain information like data, feature, threshold, left, right node
 - if it is "leaf", it will contain the predict value

```
class TreeNode:
    def __init__(self, data, depth):
        self.data = data
        self.zero = np.sum(data == 0)
        self.one = np.sum(data == 1)

    def create_node(self, feature, threshold):
        self.attr = 'node'
        self.feature = feature
        self.threshold = threshold
        self.left = None
        self.right = None

    def create_leaf(self):
        self.attr = 'leaf'
        if(self.zero > self.one):
            self.value = 0
        else:
            self.value = 1
```

DecisionTreeClassifier Class:

- **Initialization:** Initializes parameters like `max_depth` , `min_samples_leaf` , `min_gain_ratio` , and splitting criteria (`method`).

```

class DecisionTreeClassifier:
    def __init__(self, max_depth=5,min_samples_leaf=1,method='entropy'):
        self.max_depth = max_depth
        self.tree = None
        self.min_samples_leaf = min_samples_leaf
        self.min_gain_ratio = 0.01
        self.method = method

```

- **fit(X, y):** Starts the process of growing the decision tree on data `X` and target `y`.

```

def fit(self, X, y):
    y = np.array(y['label'])
    self.tree = self._grow_tree(X, y)

```

- **_grow_tree:** A recursive method that generates child nodes. If certain stopping conditions are met (e.g., reaching maximum depth, having only one unique label, or insufficient samples), it designates the node as a leaf.

```

def _grow_tree(self, X, y, depth=0):
    node = TreeNode(y, depth)
    if depth > self.max_depth or len(y) < self.min_samples_leaf or np.unique(y).shape[0] == 1:
        node.create_leaf()
        return node
    # find the best split
    feature, threshold = self.find_best_split(X, y)
    if feature is None:
        node.create_leaf()
        return node
    # split the dataset
    X_left, X_right, y_left, y_right = self.split_dataset(X, y, feature, threshold)

    # grow the tree
    node.create_node(feature, threshold)
    node.left = self._grow_tree(X_left, y_left, depth + 1)
    node.right = self._grow_tree(X_right, y_right, depth + 1)
    return node

```

- **find_best_split:** Identifies the optimal feature and threshold for splitting by iterating over features and thresholds to calculate information gain or gain ratio.

```

"""Split dataset based on a feature and threshold"""
def split_dataset(self, X, y, feature_index, threshold):

    left_indices = X[feature_index] < threshold
    right_indices = X[feature_index] >= threshold

```

```

X_left = X[left_indices]
X_right = X[right_indices]
y_left = y[left_indices]
y_right = y[right_indices]
# F1 to F17 are numeric features,
# remove the feature if it is categorical (F18~F77)
if (int(feature_index[1:])>17):
    X_left = X_left.drop(columns=[feature_index])
    X_right = X_right.drop(columns=[feature_index])

return X_left, X_right, y_left, y_right

```

- **find_best_split:** Splits the data based on a feature and threshold into left and right branches.

```

"""Find the best split for the dataset"""
def find_best_split(self, X, y):
    max_gain_ratio = float('-inf')
    best_feature, best_threshold = None, None
    for feature_index in X.columns:
        feature_values = X[feature_index].values
        for threshold in np.unique(feature_values):
            y_left, y_right = [], []
            for i in range(len(feature_values)):
                if feature_values[i] < threshold:
                    y_left.append(y[i])
                else:
                    y_right.append(y[i])
            gain_ratio = self._gain_ratio(y, y_left, y_right)
            if gain_ratio > max_gain_ratio:
                max_gain_ratio = gain_ratio
                best_feature = feature_index
                best_threshold = threshold
    """Pre-Pruning"""
    if max_gain_ratio < self.min_gain_ratio:
        return None, None
    return best_feature, best_threshold

```

- **entropy and gini:** Calculates entropy and Gini impurity, respectively.

```

def entropy(self, y):
    """Calculate the entropy of the dataset"""
    return -np.sum([(np.sum(y == c) / len(y)) * np.log2(np.sum(y == c) / len(y)) for c in np.unique(y)])
def gini(self, y):
    """Calculate the gini impurity of the dataset"""
    return 1 - np.sum([(np.sum(y == c) / len(y)) ** 2 for c in np.unique(y)])

```

- **_information_gain and _gain_ratio:** Calculate information gain and gain ratio using entropy or Gini impurity to determine the quality of a split.

```
def _information_gain(self, y, y_left, y_right):
    """Calculate the information gain of a split"""
    if self.method=='entropy':
        ig = self.entropy(y) - (len(y_left) / len(y)) * self.entropy(y_left) - (len(y_right) / len(y)) * self.entropy(y_right)
    if self.method=='gini':
        ig = self.gini(y) - (len(y_left) / len(y)) * self.gini(y_left) - (len(y_right) / len(y)) * self.gini(y_right)
    return ig

def _split_information(self, y_left, y_right):
    """Calculate Split Information."""
    n_left, n_right = len(y_left), len(y_right)
    n_total = n_left + n_right
    p_left, p_right = n_left / n_total, n_right / n_total

    # Split information (Entropy of the split proportions)
    return -sum(p * np.log2(p) for p in [p_left, p_right] if p > 0)

def _gain_ratio(self, y, y_left, y_right):
    """Calculate Gain Ratio."""
    ig = self._information_gain(y, y_left, y_right)
    si = self._split_information(y_left, y_right)

    # Avoid division by zero
    return ig / si if si != 0 else 0
```

- **predict:** Traverses the tree structure to classify each instance by routing it through features and thresholds.

```
def predict(self, X):
    if self.tree is None:
        raise ValueError("Tree is empty")
    preds = []
    for i in range(len(X)):
        preds.append(self._predict_tree(X.iloc[i], self.tree))
    return preds
```

- **print_tree:** Outputs a visual structure of the tree, with features, counts, and thresholds displayed at each level.

```
def print_tree(self, max_print_depth=3):
    if self.tree is None:
        print("Tree is empty")
        return
    self._print_tree(self.tree, max_print_depth=3)
def _print_tree(self, node, depth=0, max_print_depth=3):
    max_print_depth-=1 # because start from 0
```

```

    if node is None:
        return
    if node.attr == 'node':
        print(f'{" " * depth}[{node.feature}] [{node.zero} 0 / {node.one}]')
    else:
        print(f'{" " * depth}[{node.value}]')
    if depth >= max_print_depth or node.attr == 'leaf':
        return
    if (node.left is not None and node.right is not None):
        print(f'{" " * depth}Left:')
        self._print_tree(node.left, depth + 1)
        print(f'{" " * depth}Right:')
        self._print_tree(node.right, depth + 1)

```

Scoring System

The method `predict_score` combines these metrics into a single composite score to evaluate model performance:

$$\text{scoring} = 0.3 \times \text{accuracy} + 0.35 \times \text{f1} + 0.35 \times \text{mcc}$$

```

def predict_score(self, pred, y):
    acc = accuracy_score(pred, y)
    f1 = f1_score(pred, y, zero_division=0)
    mcc = matthews_corrcoef(pred, y)
    scoring = 0.3 * acc + 0.35 * f1 + 0.35 * mcc

    return scoring

```

Preprocessing

In this project I only use `booleanize` & `feature_selection` method to preprocess data

```

def booleanize(self):
    # Booleanize the DataFrame

    # Boolean features (F18-F77):
    # F18 to F67, F60 to F72 are categorical features, each with a value 'Yes'
    # F68 with a value 'Male' or 'Female',
    # F73 to F77 with a value 'Infected' or 'Non-infected'.
    for i in range(18, 68):
        self.df.iloc[:, i] = self.df.iloc[:, i].apply(lambda x: 1 if x == 'Yes' else 0)
    self.df.iloc[:, 68] = self.df.iloc[:, 68].apply(lambda x: 1 if x == 'Male' else 0)
    for i in range(69, 73):
        self.df.iloc[:, i] = self.df.iloc[:, i].apply(lambda x: 1 if x == 'Yes' else 0)
    for i in range(73, 78):
        self.df.iloc[:, i] = self.df.iloc[:, i].apply(lambda x: 1 if x == 'Infected' else 0)

```

```

        return self.df
def feature_selection(self, threshold):
    # Return the correlation between features
    # find absolute correlation >= threshold
    column = self.df.corr()['label'][abs(self.df.corr()['label']) >= thresh
    # get the column name
    column = column.tolist()
    return column

```

Main.py

```

def main():
    train_X, train_y, test_X = dataPreprocessing(0.1) # train, test data shou

    # split the training data into training and validation data
    total = len(train_X)
    train_size = int(total * 0.8)
    train_X, val_X = train_X[:train_size], train_X[train_size:]
    train_y, val_y = train_y[:train_size], train_y[train_size:]

    params = {
        'max_depth': [15, 20, 25],
        'min_samples_leaf': [1, 3, 5],
        'method': ['entropy', 'gini']
    }
    # use KFold_cross_validation to find the best hyperparameters

    #best_params, best_score = find_best_params(train_X, train_y, params)
    #model = DecisionTreeClassifier(max_depth=best_params['max_depth'], min_s

    # best model
    model = DecisionTreeClassifier(max_depth=25, min_samples_leaf=5, method='
    model.fit(train_X, train_y)

    pred = model.predict(val_X)
    scoring = model.predict_score(pred, val_y)
    print(f'Scoring: {scoring:.5f}')
    print('-' * 50)

    # post prune the tree with validation data
    print('After pruning\n\n')
    model.post_prune(val_X, val_y)

    pred = model.predict(val_X)
    scoring = model.predict_score(pred, val_y)
    print(f'Scoring: {scoring:.5f}')

```

```

print('-' * 50)
model.print_tree()
print('-' * 50)

# predict the output of the testing data
# remember to save the predict label as .csv file
test_pred = model.predict(test_X)
df = pd.DataFrame()
df['label'] = test_pred
df = pd.DataFrame()
df['label'] = pred
root_path = './' # change the root path
df.to_csv(root_path+'pred_hw3.csv')
print('Predict label has been saved as pred_hw3.csv')

```

Problem with Using Information Gain for Splitting

Q: What is the problem if you use Information Gain to split data in your decision tree model?

- When using Information Gain (IG) for splitting, one common issue is its tendency to favor features with many distinct values. These high-cardinality features, while providing excellent separation of the training data, may not generalize well to new, unseen data, leading to overfitting.
- For instance, a feature with many unique values(ex: id of the input data) for each sample would always provide high IG but might not generalize well on new data.

Solution:

To address this, you can use **Gain Ratio** instead of IG. **Gain Ratio** takes into account the "**split information**" - the entropy of the feature's value distribution. By dividing the Information Gain by the split information, **Gain Ratio penalizes features with many distinct values**, favoring splits that provide a better balance between predictive power and generalization.

The Gain Ratio formula is:

$$\text{Gain Ratio} = \frac{\text{Information Gain}}{\text{Split Information}}$$

Using Gain Ratio helps the decision tree model focus on selecting splits that improve performance on new data, rather than just optimizing for the training set.

Tree Structure Screenshot and Explanation

Tree Structure

```
-----  
[F5] [247 0 / 93 1] [Threshold = 77.2160902102211]  
Left:  
  [1]  
Right:  
  [F5] [247 0 / 91 1] [Threshold = 199.2160902102211]  
    Left:  
      [F7] [247 0 / 89 1] [Threshold = 160.4825708615861]  
        Left:  
          [F8] [247 0 / 87 1] [Threshold = 41.43019958202489]  
            Right:  
              [1]  
        Right:  
          [1]  
-----
```

Explanation of Tree Structure

Each line in the printed structure provides information about a node:

- **[Feature]:** The feature used for splitting.
- **[Count of Labels]:** Shows the distribution of zero and one labels in the node.
- **[Threshold]:** The threshold for the feature to split data into left and right branches.
- **Left and Right:** Indicate child nodes; if the node is a leaf, it outputs a single value indicating the class prediction.

Discussion

Strategy to find the best model

The strategy for finding the best model uses `KFold_cross_validation` and `find_best_params`.

1. `KFold_cross_validation` splits the data into `k` folds, trains a `DecisionTreeClassifier` on each fold, applies post-pruning, and calculates metrics like accuracy, F1, and MCC, combining them into a composite score for balanced evaluation.
2. `find_best_params` then performs a grid search over hyperparameters (`max_depth`, `min_samples_leaf`, `method`). It uses cross-validation to find the optimal combination that maximizes the composite score, tracking the best `scoring` and parameters.
3. The best parameters are used to train a model on the full dataset, ready for final testing evaluation. This approach optimizes performance while avoiding overfitting.
4. Get the best params (`max_depth`=25, `min_samples_leaf`=5, `method`='gini')


```

params = {
    'max_depth': [15, 20, 25],
    'min_samples_leaf': [1, 3, 5],
    'method': ['entropy', 'gini']
}

def KFold_cross_validation(X, y, k, max_depth=25, min_samples_leaf=3, method=
    fold_size = len(X) // k
    data = pd.concat([X, y], axis=1)
    data = data.sample(frac=1).reset_index(drop=True)

    scores = {
        'accuracy': [],
        'f1': [],
        'precision': [],
        'recall': [],
        'mcc': [],
        'roc_auc': []
    }

    for i in tqdm(range(k)):
        start, end = i * fold_size, (i + 1) * fold_size
        val_data = data[start:end]
        train_data = pd.concat([data[:start], data[end:]])

        X_train, y_train = train_data.iloc[:, :-1], pd.DataFrame(train_data.il
        X_val, y_val = val_data.iloc[:, :-1], val_data.iloc[:, -1]

        model = DecisionTreeClassifier(max_depth, min_samples_leaf, method)
        model.fit(X_train, y_train)
        # post-pruning
        model.post_prune(X_val, y_val)
        y_pred = model.predict(X_val)

        scores['accuracy'].append(accuracy_score(y_val, y_pred))
        scores['f1'].append(f1_score(y_val, y_pred))
        scores['precision'].append(precision_score(y_val, y_pred))
        scores['recall'].append(recall_score(y_val, y_pred))
        scores['mcc'].append(matthews_corrcoef(y_val, y_pred))
        scores['roc_auc'].append(roc_auc_score(y_val, y_pred))
        #scores['score'] = model.predict_score(y_pred, y_val)

    for metric in scores:
        scores[metric] = np.mean(scores[metric])
    scoring = 0.3 * scores['accuracy'] + 0.35 * scores['f1'] + 0.35 * scores[
    scores['scoring'] = scoring
    print('scoring:', scoring)
    print('\n')

```

```

    return scores

def find_best_params(X, y, params):
    best_params = {
        'max_depth': None,
        'min_samples_leaf': None,
        'method': None,
    }
    best_score = 0
    for max_depth in params['max_depth']:
        for min_samples_leaf in params['min_samples_leaf']:
            for method in params['method']:
                print('max_depth:', max_depth, 'min_samples_leaf:', min_samples_leaf, 'method:', method)
                score = KFold_cross_validation(X, y, 10, max_depth, min_samples_leaf, method)
                if score['scoring'] > best_score:
                    best_score = score['scoring']
                    best_params = {
                        'max_depth': max_depth,
                        'min_samples_leaf': min_samples_leaf,
                        'method': method
                    }
    print('best_params:', best_params, 'best_score:', best_score)
    return best_params, best_score

```

Bonus

Additional Features

- **Pre-Pruning:** A method helps prevent the tree from growing too deep and complex, potentially overfitting the training data. By setting an appropriate `min_gain_ratio` value, you can control the trade-off between tree complexity and model performance.

```

"""Find the best split for the dataset"""
def find_best_split(self, X, y):
    # Same as the code in find_best_split
    """Pre-Pruning"""
    if max_gain_ratio < self.min_gain_ratio:
        return None, None
    return best_feature, best_threshold

```

- **Post-Pruning:** A method that evaluates nodes on validation data. If making a node a leaf improves model performance, it prunes that branch.

```

# post-pruning
def post_prune(self, validation_X, validation_y):

```

```

    if self.tree is None:
        raise ValueError("Tree is empty")
    self.prune_node(self.tree, validation_X, validation_y)
def prune_node(self, node, validation_X, validation_y):
    if node.attr == 'leaf':
        return
    # Calculate current performance
    original_score = self.predict_score(self.predict(validation_X), valida

    # Try making node a leaf
    temp_attr = node.attr
    node.create_leaf()

    # Calculate new performance
    new_score = self.predict_score(self.predict(validation_X), validation_

    # Revert if no improvement
    if new_score <= original_score:
        node.attr = temp_attr
    else:
        print(f'Pruned node: {node.feature} {node.threshold}')
    # Continue pruning children if not leaf
    if node.attr != 'leaf':
        self.prune_node(node.left, validation_X, validation_y)
        self.prune_node(node.right, validation_X, validation_y)

```